

1. prim1

2. Kruskal6

3. Dijkstra14

4. Floyd20

5. DFS23

6. Huffman26

1. Prim

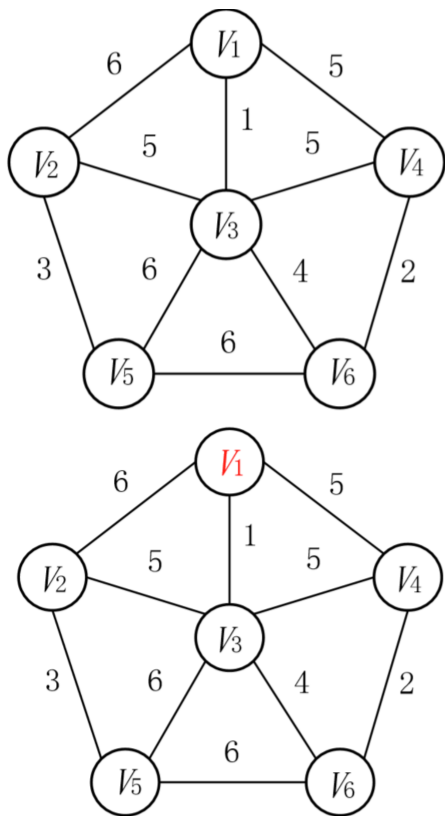
算法简单描述

- 1).输入：一个加权连通图，其中顶点集合为V，边集合为E；
- 2).初始化： $V_{new} = \{x\}$ ，其中x为集合V中的任一节点（起始点）， $E_{new} = \{\}$,为空；
- 3).重复下列操作，直到 $V_{new} = V$ ：

a.在集合E中选取权值最小的边 $\langle u, v \rangle$ ，其中u为集合V中的元素，而v不在 V_{new} 集合当中，并且 $v \in V$ （如果存在有多条满足前述条件即具有相同权值的边，则可任意选取其中之一）；

b.将v加入集合 V_{new} 中，将 $\langle u, v \rangle$ 边加入集合 E_{new} 中；
- 4).输出：使用集合 V_{new} 和 E_{new} 来描述所得到的最小生成树。

图例



已选的点集 $V_{new} = \{ \}$

已选的边集 $E_{new} = \{ \}$

当前可选点集 $U = \{ \}$

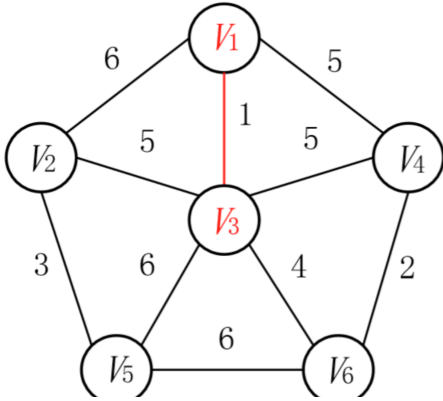
当前所选边的权值总和：0

已选的点集 $V_{new} = \{ V_1 \}$

已选的边集 $E_{new} = \{ \}$

当前可选点集 $U = \{ V_2, V_3, V_4 \}$

当前所选边的权值总和：0

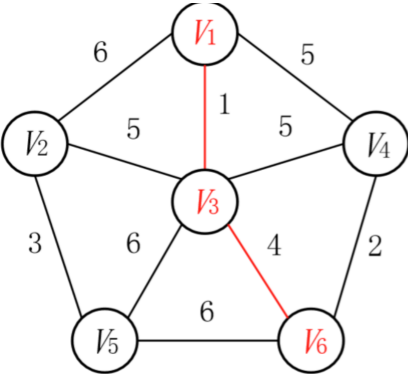


已选的点集 $V_{\text{new}} = \{ V_1, V_3 \}$

已选的边集 $E_{\text{new}} = \{ \langle V_1, V_3 \rangle \}$

当前可选点集 $U = \{ V_2, V_4, V_5, V_6 \}$

当前所选边的权值总和：1

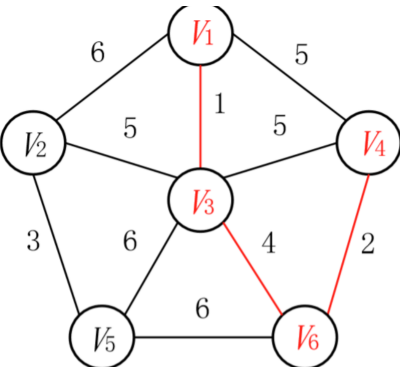


已选的点集 $V_{\text{new}} = \{ V_1, V_3, V_6 \}$

已选的边集 $E_{\text{new}} = \{ \langle V_1, V_3 \rangle, \langle V_3, V_6 \rangle \}$

当前可选点集 $U = \{ V_2, V_4, V_5 \}$

当前所选边的权值总和：5

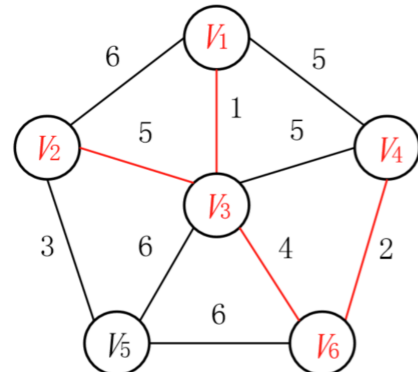


已选的点集 $V_{\text{new}} = \{ V_1, V_3, V_4, V_6 \}$

已选的边集 $E_{\text{new}} = \{ \langle V_1, V_3 \rangle, \langle V_3, V_6 \rangle, \langle V_6, V_4 \rangle \}$

当前可选点集 $U = \{ V_2, V_5 \}$

当前所选边的权值总和：7

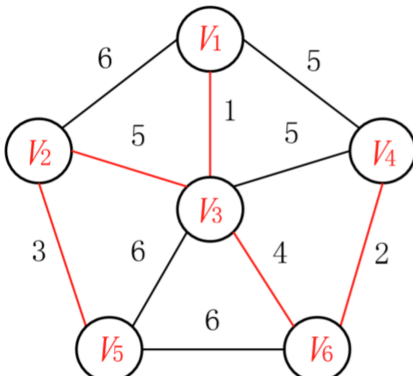


已选的点集 $V_{\text{new}} = \{ V_1, V_2, V_3, V_4, V_6 \}$

已选的边集 $E_{\text{new}} = \{ \langle V_1, V_3 \rangle, \langle V_3, V_6 \rangle, \langle V_6, V_4 \rangle, \langle V_3, V_2 \rangle \}$

当前可选点集 $U = \{ V_5 \}$

当前所选边的权值总和：12



已选的点集 $V_{\text{new}} = \{ V_1, V_2, V_3, V_4, V_5, V_6 \}$

已选的边集 $E_{\text{new}} = \{ \langle V_1, V_3 \rangle, \langle V_3, V_6 \rangle, \langle V_6, V_4 \rangle, \langle V_3, V_2 \rangle, \langle V_2, V_5 \rangle \}$

当前可选点集 $U = \{ \}$

当前所选边的权值总和：15

我们把一个整图定义为Graph。这里还需要定义3个额外的辅助数组：

key[n]: 用来表示最小生成树中的各个顶点与集合中的顶点最短边的权值。初始化为无穷大, 初始点key[0] = 0;

parent[n]: 用来存储最小生成树。孩子父亲表示法, 初始值为-1, 表示还没有父亲节点。

mstSet[n]: 标记数组, 用于标记加入边对应的顶点。初始化均为false。

代码思想:

1. 首先初始化 3 个辅助数组, key=INF (表示无穷大) , parent=-1 , mstSet=False.
2. key[0]=0, 表示以0为最开始的顶点到自生距离为0.
3. 执行一下n-1次循环。

在key数组中选取最小边, key[k], 去找到相应的顶点k。并标记顶点mstSet[u] = true;

将以k为起点的k->v, 和key[i]里面的边进行比较【i满足mstSet[i] == false】;

如果graph(k)(i) < key[i] 更新key[i] = graph(k)(i), 表示将k->i的边加入到最小生成树;

代码实现

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

int numVertices, numEdges;

int minKey(vector<int> key, vector<bool> mstSet) {
    // 找到key数组最小的边的序号
    int min = INT_MAX, min_index;

    for (int v = 0; v < key.size(); v++) {
        if (mstSet[v] == false && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }

    return min_index;
}

int printMST(vector<int> parent, vector<vector<int>>> graph) {
    cout << "最小生成树的边: " << endl;
    int sum = 0;
    for (int i = 1; i < parent.size(); i++) {
        cout << parent[i] << " - " << i << "    权重: " << graph[i][parent[i]]
        << endl;
        sum += graph[i][parent[i]];
    }
}
```

```

        cout << "\n最小生成树的代价：" << sum << endl;
        return sum;
    }

    void primMST(vector<vector<int>> graph) {

        vector<int> parent(numVertices);
        vector<int> key(numVertices, INT_MAX);
        vector<bool> mstSet(numVertices, false); //初始化标记数组mstSet

        key[0] = 0;
        parent[0] = -1; //存储最小生成树，用孩子父亲表示法

        for (int count = 0; count < numVertices - 1; count++) {
            int u = minKey(key, mstSet);
            mstSet[u] = true;

            for (int v = 0; v < numVertices; v++) {
                if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                    parent[v] = u;
                    key[v] = graph[u][v];
                }
            }
        }

        printMST(parent, graph);
    }

    int main() {
        /*
        6 10
        0 1 6
        0 2 1
        0 3 5
        1 2 5
        1 4 3
        2 3 5
        2 4 6
        2 5 4
        3 5 2
        4 5 6
        */
        /*
        vector<vector<int>> graph = {
            {0, 2, 0, 6, 0},
            {2, 0, 3, 8, 5},
            {0, 3, 0, 0, 7},
            {6, 8, 0, 0, 9},
            {0, 5, 7, 9, 0}
        };
        */

        // 输入顶点和边的数量
        cout << "请输入顶点数量：";
        cin >> numVertices;
    }

```

```

    cout << "请输入边的数量: ";
    cin >> numEdges;

    vector<vector<int>> graph(numVertices, vector<int>(numVertices, 0));
    //numVertex * numBertices的二维矩阵存图

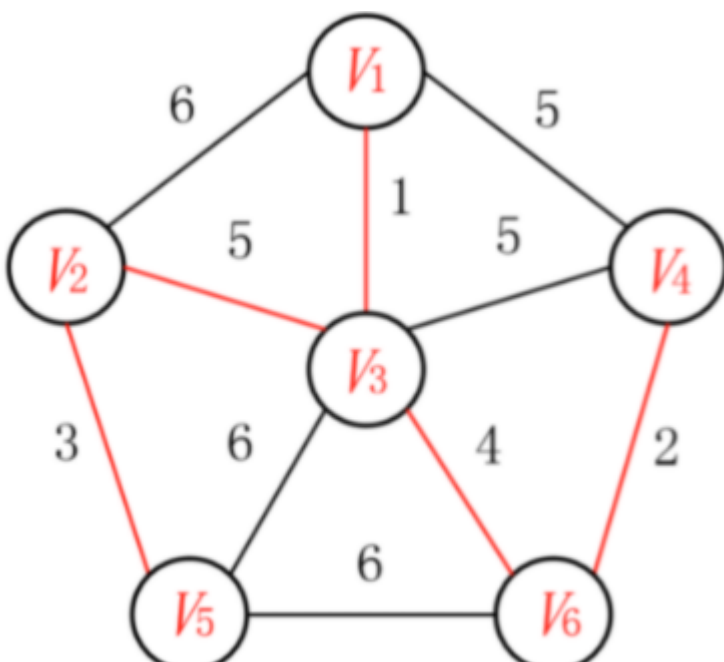
    // 输入每条边的信息
    cout << "请输入每条边的信息 ( 起点 终点 权重 ):" << endl;
    for (int i = 0; i < numEdges; i++) {
        int src, dest, weight;
        cin >> src >> dest >> weight;
        graph[src][dest] = weight;
        graph[dest][src] = weight; // 如果是无向图，需要考虑双向边
    }

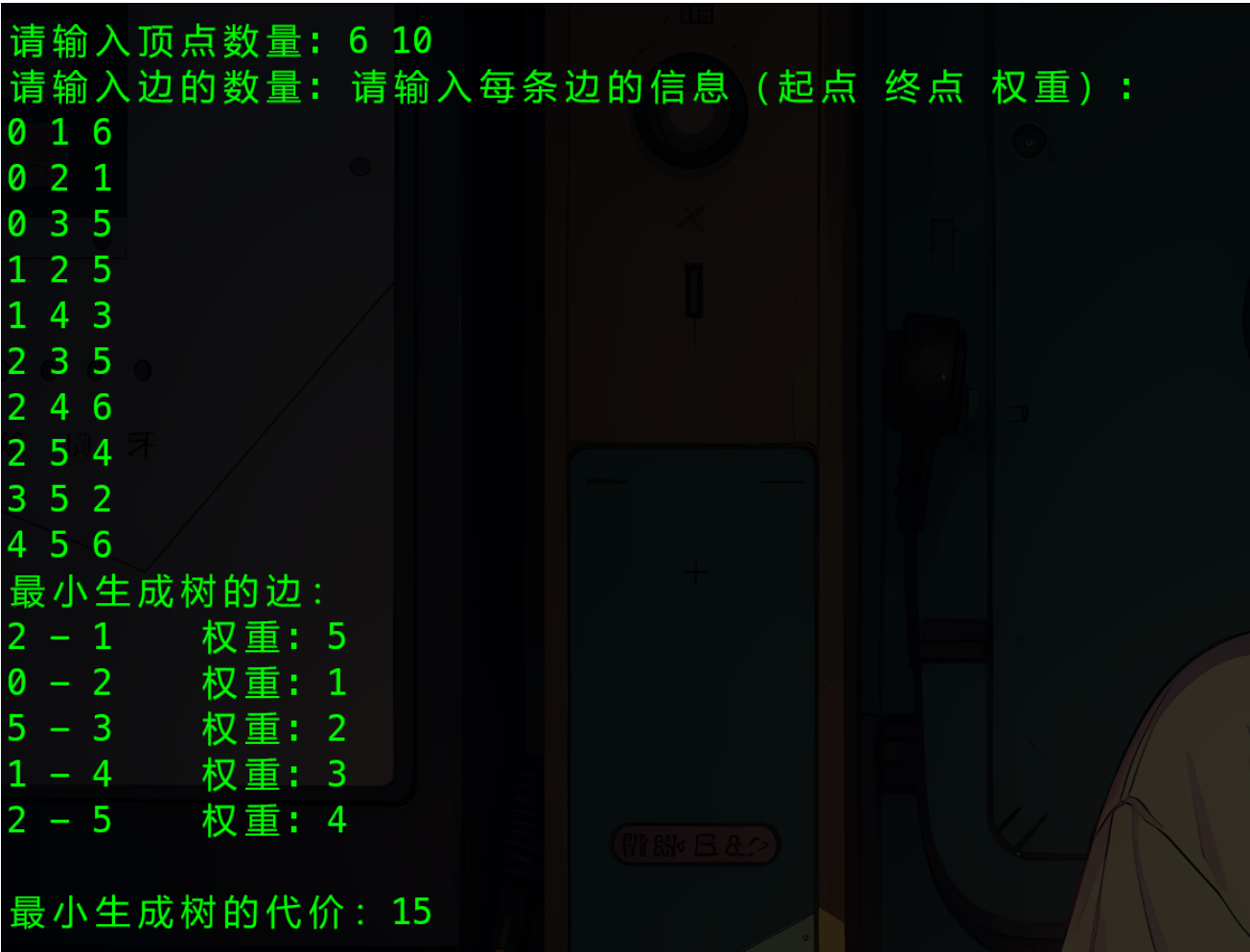
    // 模拟输出
    primMST(graph);

    return 0;
}

```

输入图例





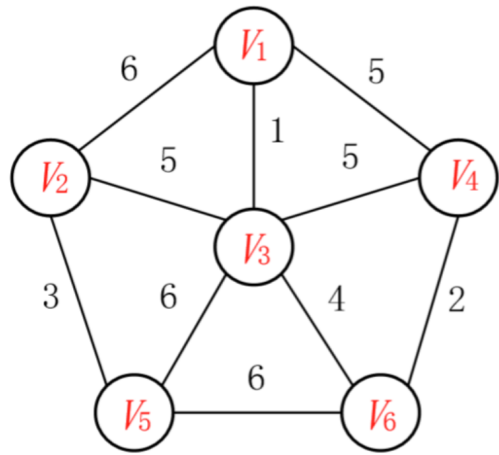
2. Kruskal

算法简单描述：

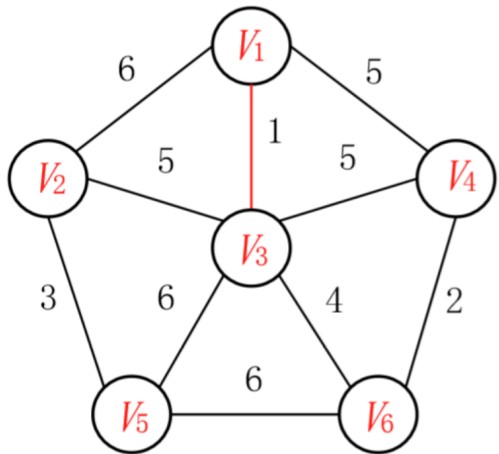
1. 设无向连通图 $Graph$ 有 v 个顶点， e 条边；
2. 新建图 $Graph_{new}$ ， $Graph_{new}$ 拥有与原图中相同的 v 个顶点，但没有边；
3. 将原图 $Graph$ 中所有边按权值从小到大排序；
4. 进入一层循环，该循环从权值最小的边开始遍历每条边，直至图 $Graph_{new}$ 中所有的节点都在同一个连通分量中。循环体内的内容如下： if（这条边连接的两个节点于图 $Graph_{new}$ 中不在同一个连通分量中）则添加这条边到图 $Graph_{new}$ 中；

图例

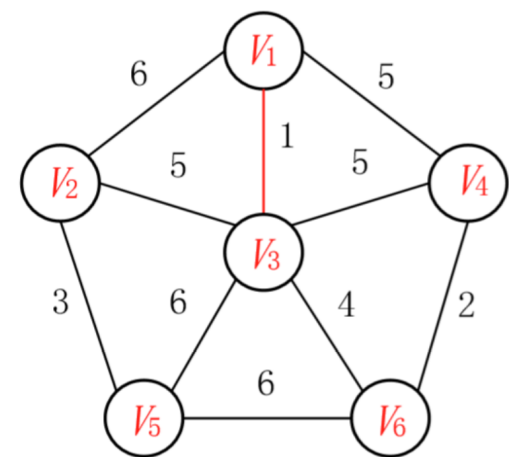
序号	边	长度
1	$\langle V_1, V_3 \rangle$	1
2	$\langle V_4, V_6 \rangle$	2
3	$\langle V_2, V_5 \rangle$	3
4	$\langle V_3, V_6 \rangle$	4
5	$\langle V_1, V_4 \rangle$	5
6	$\langle V_2, V_3 \rangle$	5
7	$\langle V_3, V_4 \rangle$	5
8	$\langle V_1, V_2 \rangle$	6
9	$\langle V_3, V_5 \rangle$	6
10	$\langle V_5, V_6 \rangle$	6



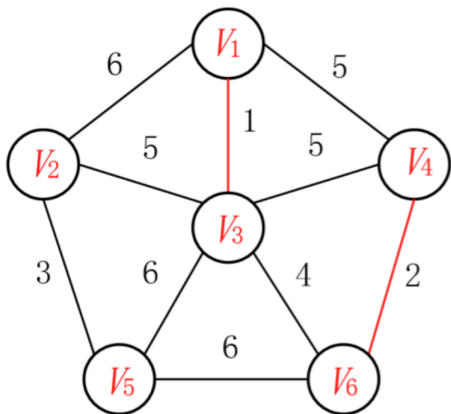
已选边集 $E = \{ \}$



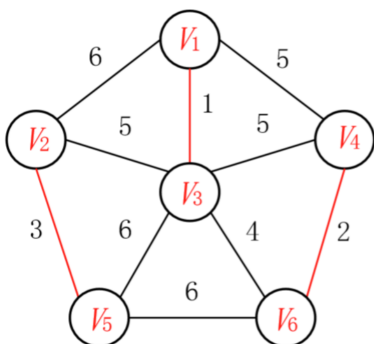
已选边集 $E = \{ \langle V_1, V_3 \rangle \}$



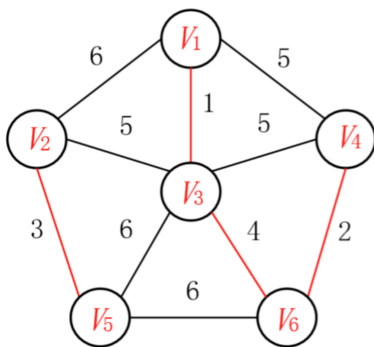
已选边集 $E = \{ \langle V_1, V_3 \rangle \}$



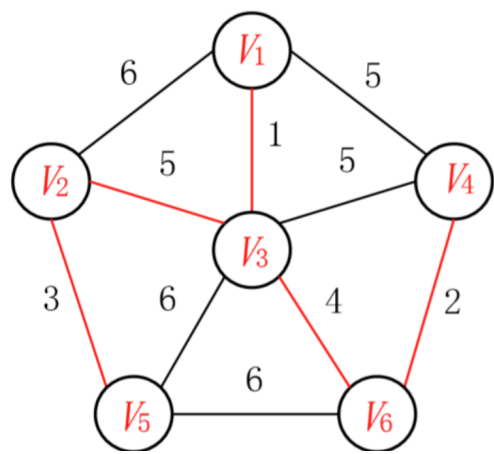
已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle, \langle V_3, V_6 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle, \langle V_3, V_6 \rangle, \langle V_2, V_3 \rangle \}$

1.数据结构定义：

Edge 结构体表示图中的一条边，包括起始顶点、目标顶点和边的权重。

Graph 结构体表示整个图，包括顶点数（V）、边数（E）和边的列表。

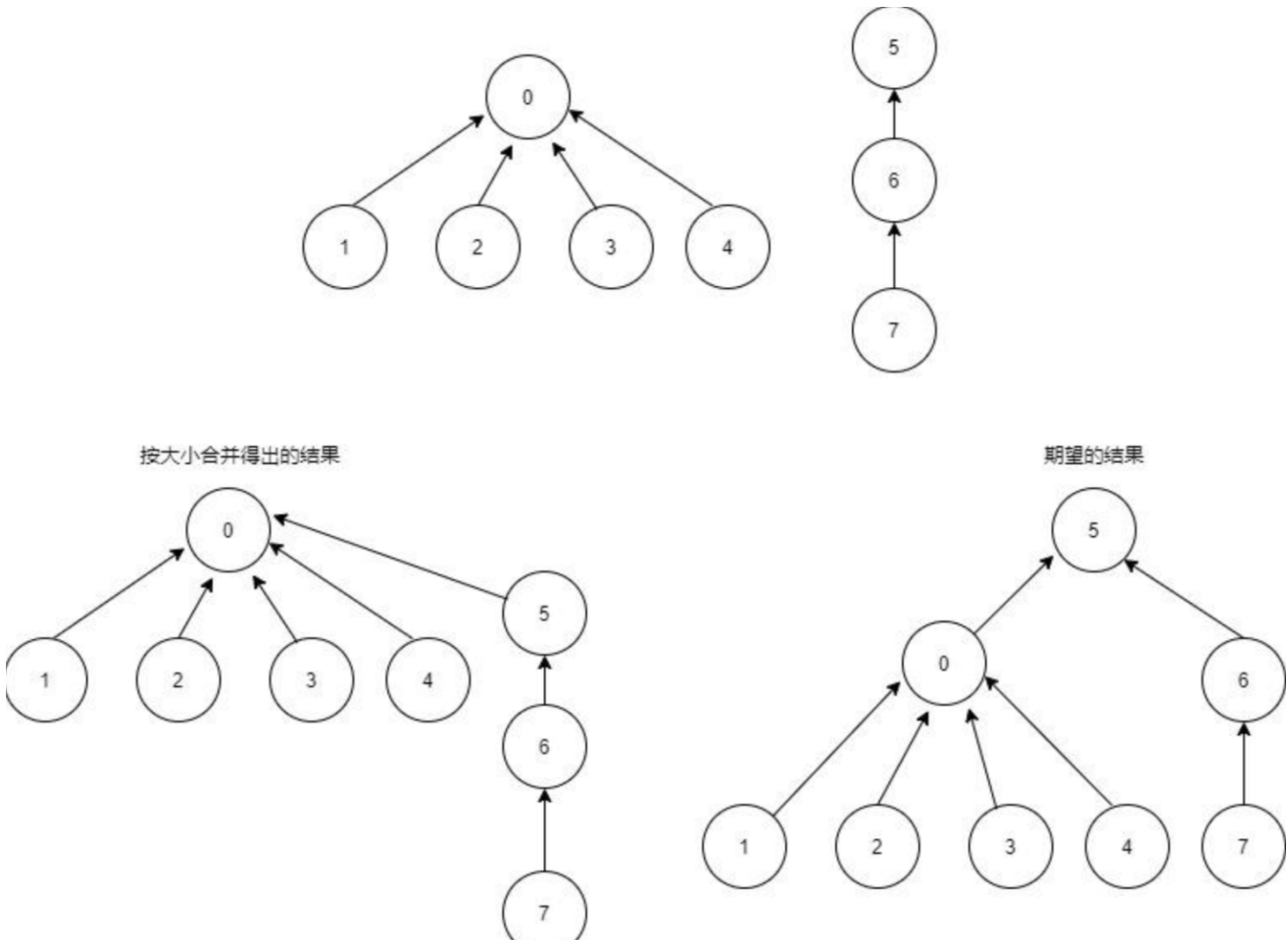
2.并查集定义：

使用并查集来检测图中是否存在环。每个顶点都属于一个集合，初始时每个集合只包含一个顶点。

DisjointSet类表示并查集，包括父节点和秩（rank）。以及查找父节点和聚合方法。

```
if (rank[rootU] < rank[rootV]) {  
    // U的高度<V的高度， 将U指向V, U,V的高度就不会变化，反之V的高度会增加，增加查找时间  
    parent[rootU] = rootV;  
}
```

如果 U的高度< V的高度， 将U指向V, U,V的高度就不会变化，反之V的高度会增加，增加查找时间



3.排序边：

边按照权重从小到大排序，以便后续选择最小权重的边。

4.Kruskal算法主体：

遍历排好序的边，逐个考虑添加到最小生成树中。对于每条边，判断边的两个顶点是否在同一个集合中。如果不在同一个集合中，说明加入这条边不会形成环，可以添加到最小生成树中。如果两个顶点在不同的集合中，将它们合并为一个集合。

重复上述步骤，直到最小生成树中包含了图中的所有顶点（顶点数-1 条边）。

5.输出最小生成树的边：

最后输出构成最小生成树的所有边及其权重。

总体来说，Kruskal算法按照边的权重从小到大的顺序选择边，然后通过并查集来检测是否形成环。如果不形成环，就将这条边添加到最小生成树中。这样逐步选择边，直到最小生成树构建完成

代码实现

c++:

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```

using namespace std;

// 存储边信息
struct Edge {
    int src, dest, weight;
};

class Graph {
public:
    int V, E;
    vector<Edge> edges;

    Graph(int v, int e) : V(v), E(e) {}

    void addEdge(int src, int dest, int weight) {
        Edge edge = { src, dest, weight };
        edges.push_back(edge);
    }

    vector<Edge> kruskalMST();
};

class DisjointSet {
    /**
     * 并查集
     * 找祖先
     *
     */
public:
    vector<int> parent, rank;

    DisjointSet(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) {
            parent[i] = i; //初始化自己指向自己
        }
    }

    int find(int u) {
        if (u != parent[u]) {
            parent[u] = find(parent[u]);
        }
        return parent[u];
    }

    void unionSets(int u, int v) {
        int rootU = find(u);
        int rootV = find(v);
        /**
         * rank数组记录每颗树的每个节点的高度
         * 节省查找祖先的时间
         */
        if (rank[rootU] < rank[rootV]) {
            // U的高度<V的高度，将U指向V，U,V的高度就不会变化，反之V的高度会增加，
            增加查找时间
            parent[rootU] = rootV;
        }
    }
};

```

```
        }
        else if (rank[rootU] > rank[rootV]) {
            parent[rootV] = rootU;
        }
        else {
            parent[rootU] = rootV;
            rank[rootV]++;
        }
    }
};

bool compareEdges(Edge a, Edge b) {
    // 排序规则
    return a.weight < b.weight;
}

vector<Edge> Graph::kruskalMST() {
    vector<Edge> result;

    // 结构体存储边信息进行排序，排序规则compareEdges， weight的升序
    sort(edges.begin(), edges.end(), compareEdges);

    // Step 2: Create a disjoint set to track components
    DisjointSet ds(V);
    // cout << "V: " << V << endl;
    // Step 3: Iterate through all sorted edges
    for (Edge edge : edges) {
        int srcRoot = ds.find(edge.src);
        int destRoot = ds.find(edge.dest);

        // 如果不是同一个祖先就加入
        if (srcRoot != destRoot) {
            result.push_back(edge);
            ds.unionSets(srcRoot, destRoot);
        }
    }

    return result;
}

int main() {
    /**
        6 10
        0 1 6
        0 2 1
        0 3 5
        1 2 5
        1 4 3
        2 3 5
        2 4 6
        2 5 4
        3 5 2
        4 5 6
    */
    int numVertices, numEdges;

    // 输入顶点和边的数量
```

```

cout << "请输入顶点数量: ";
cin >> numVertices;
cout << "请输入边的数量: ";
cin >> numEdges;

Graph graph(numVertices, numEdges);

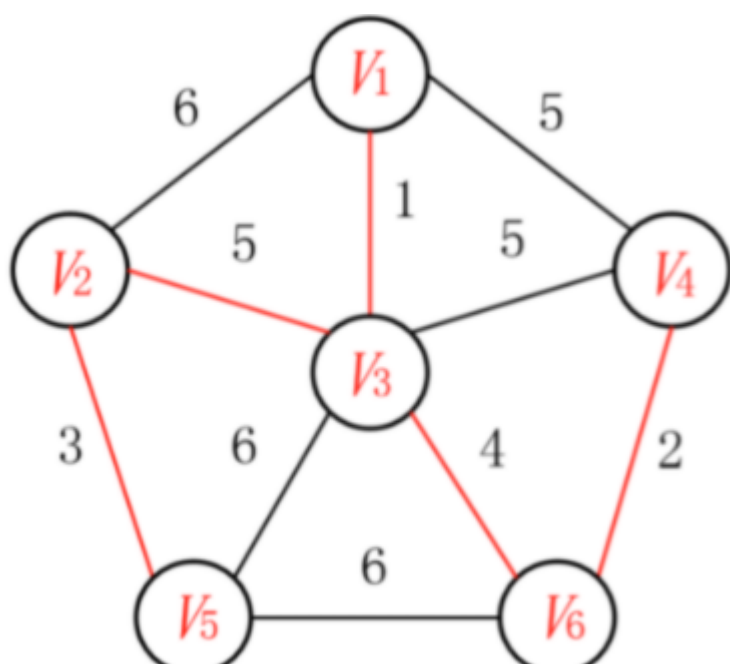
// 输入每条边的信息
cout << "请输入每条边的信息 ( 起点 终点 权重 ):" << endl;
for (int i = 0; i < numEdges; i++) {
    int src, dest, weight;
    cin >> src >> dest >> weight;
    graph.addEdge(src, dest, weight); //将边加入
}

// 执行Kruskal算法
vector<Edge> mst = graph.kruskalMST();

// 输出最小生成树的边和权重
cout << "最小生成树的边:" << endl;
int sum = 0;
for (Edge edge : mst) {
    cout << edge.src << " - " << edge.dest << "    权重:" << edge.weight
<< endl;
    sum += edge.weight;
}
cout << "\n最小生成树的代价:" << sum << endl;
return 0;
}

```

输入图例:





3. Dijkstra

算法简单描述

Dijkstra(迪杰斯特拉)算法是典型的单源最短路径算法，用于计算一个节点到其他所有节点的最短路径。主要特点是以起始点为中心向外层层扩展，直到扩展到终点为止。Dijkstra算法是很有代表性的最短路径算法，在很多专业课程中都作为基本内容有详细的介绍，如数据结构，图论，运筹学等等。注意该算法要求图中不存在**负权边**。

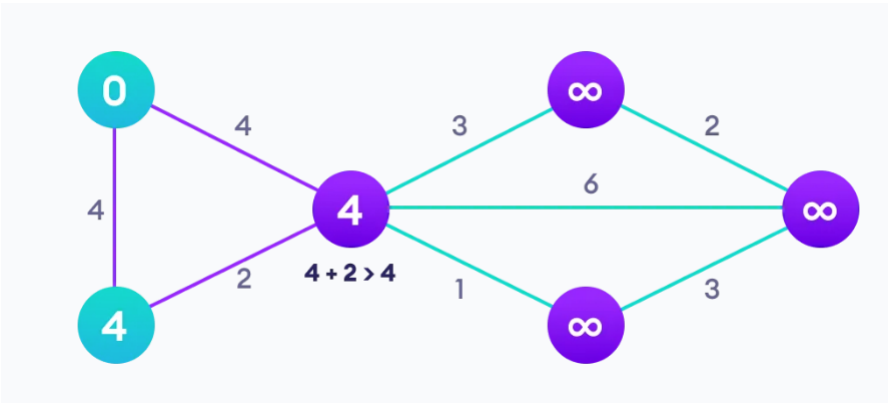
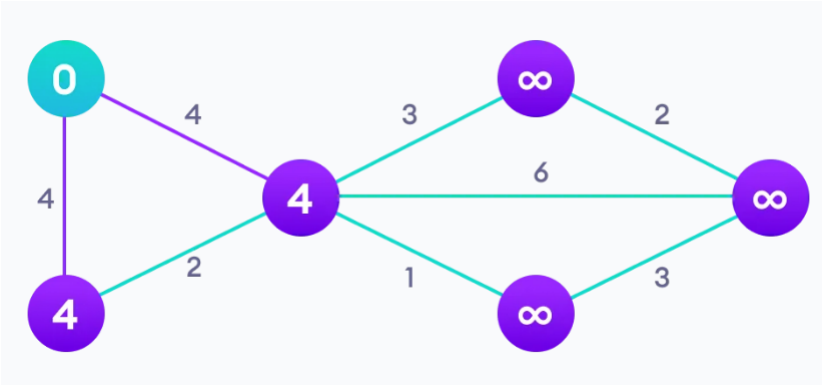
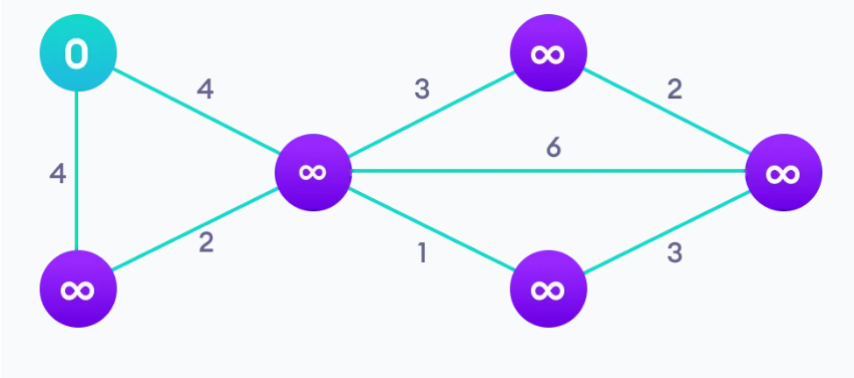
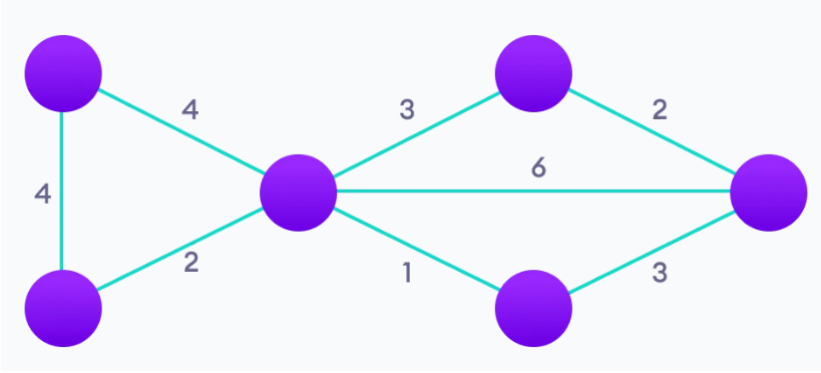
问题描述：在无向图 $G=(V,E)$ 中，假设每条边 $E[i]$ 的长度为 $w[i]$ ，找到由顶点 V_0 到其余各点的最短路径。（单源最短路径）

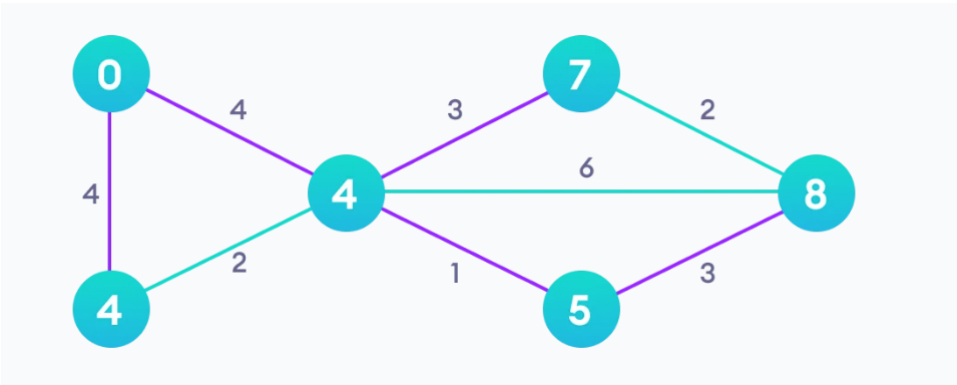
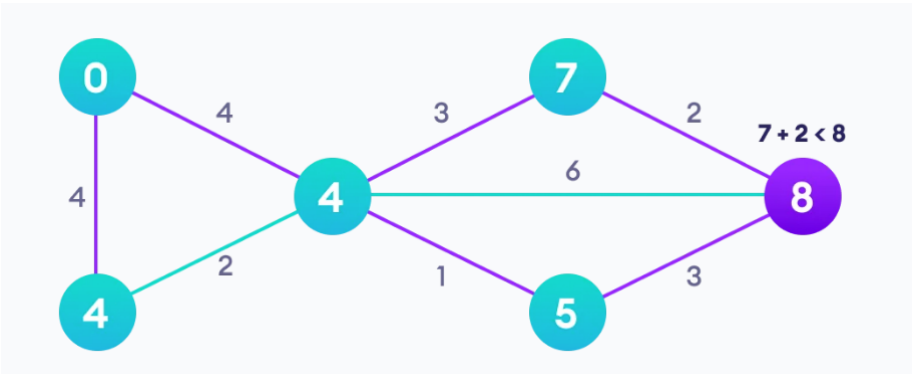
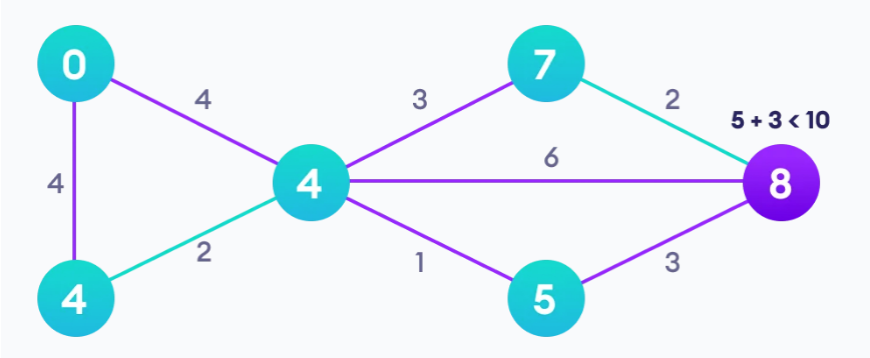
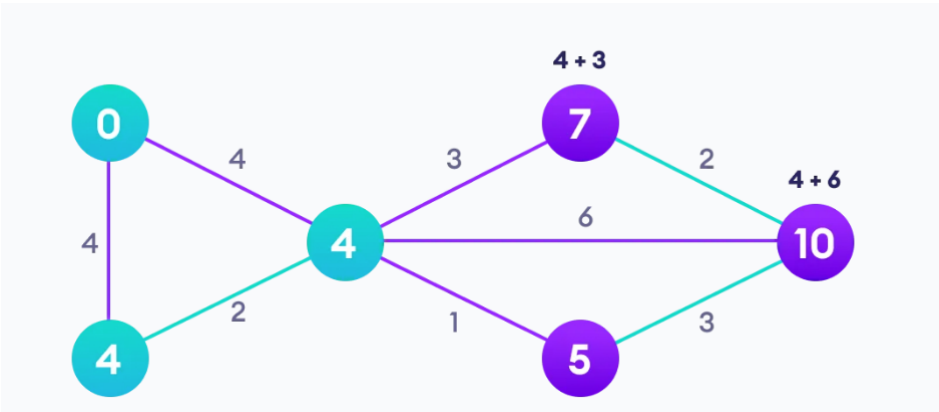
1)算法思想：设 $G=(V,E)$ 是一个带权有向图，把图中顶点集合 V 分成两组，第一组为已求出最短路径的顶点集合（用 S 表示，初始时 S 中只有一个源点，以后每求得一条最短路径，就将加入到集合 S 中，直到全部顶点都加入到 S 中，算法就结束了），第二组为其余未确定最短路径的顶点集合（用 U 表示），按最短路径长度的递增次序依次把第二组的顶点加入 S 中。在加入的过程中，总保持从源点 v 到 S 中各顶点的最短路径长度不大于从源点 v 到 U 中任何顶点的最短路径长度。此外，每个顶点对应一个距离， S 中的顶点的距离就是从 v 到此顶点的最短路径长度， U 中的顶点的距离，是从 v 到此顶点只包括 S 中的顶点为中间顶点的当前最短路径长度。

2)算法步骤：

- a.初始时， S 只包含源点，即 $S = \{v\}$ ， v 的距离为0。 U 包含除 v 外的其他顶点，即： $U=\{\text{其余顶点}\}$ ，若 v 与 U 中顶点 u 有边，则 $\langle u,v \rangle$ 正常有权值，若 u 不是 v 的出边邻接点，则 $\langle u,v \rangle$ 权值为 ∞ 。
- b.从 U 中选取一个距离 v 最小的顶点 k ，把 k ，加入 S 中（该选定的距离就是 v 到 k 的最短路径长度）。
- c.以 k 为新考虑的中间点，修改 U 中各顶点的距离；若从源点 v 到顶点 u 的距离（经过顶点 k ）比原来距离（不经过顶点 k ）短，则修改顶点 u 的距离值，修改后的距离值的顶点 k 的距离加上边上的权。
- d.重复步骤b和c直到所有顶点都包含在 S 中。

图例





代码实现

堆优化

堆优化的主要思想就是使用一个优先队列（就是每次弹出的元素一定是整个队列中最小的元素）来代替最近距离的查找，用邻接表代替邻接矩阵，这样可以大幅度节约时间开销，就是对下面的代码进行优化，使用优先队列，就不用每次 $O(n)$ 查找，效率 $O(n\log n)$ 。


```

#include <iostream>
#include <vector>
#include <queue>
#include <climits>

using namespace std;

const int INF = INT_MAX; // 无穷大表示不可达

// 图的邻接矩阵表示
vector<vector<int>>> graph;

// Dijkstra 算法
void dijkstra(int start, vector<int>& distance, vector<int>& parent) {
    /**
     * start: 起点
     * distance: 距离数组
     * parent: 记录路径的数组
     */
    int n = graph.size();
    distance.resize(n, INF);
    parent.resize(n, -1);

    /**
     * pair<int, int> 是队列中的元素类型，表示一个包含两个整数的对，第一个整数是距离，第二个整数是节点索引
     * vector<pair<int, int>> 是底层容器的类型，表示存储元素的容器，这里使用了vector。
     * greater<pair<int, int>> 是比较函数对象，用于确定队列中元素的优先级。greater是一个模板类，它实现了逆序比较，即按照降序排列。
     * 这里它用于确保队列中的元素按照距离的升序排列，使得队首元素是距离最小的节点
     */
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    pq.push({0, start});
    distance[start] = 0;

    while (!pq.empty()) {
        int u = pq.top().second;
        int dist_u = pq.top().first;
        pq.pop();

        // 如果当前节点已经更新过了，跳过
        if (dist_u > distance[u]) {
            continue;
        }

        for (int v = 0; v < n; ++v) {
            if (graph[u][v] != INF) {
                int alt = distance[u] + graph[u][v];
                if (alt < distance[v]) {
                    distance[v] = alt;
                    //记录最短路径: start->v 是从start->...->u->v, 这里记录的是v->u, 最后需要逆序遍历

```

```

        parent[v] = u;
        pq.push({distance[v], v});
    }
}

// 输出最短路径
void printShortestPath(int start, int end, const vector<int>& distance, const
vector<int>& parent) {
    vector<int> path;
    for (int v = end; v != -1; v = parent[v]) {
        path.push_back(v);
    }

    cout << "路径 : ";
    // 逆序遍历
    for (int i = path.size() - 1; i >= 0; --i) {
        cout << path[i];
        if (i != 0) {
            cout << " -> ";
        }
    }
    cout << "\t最短路径长度 : " << distance[end] << endl;
}

/**
 * 0序
 *
 * 6 8
 * 0 1 4
 * 0 2 4
 * 1 2 2
 * 2 3 3
 * 2 4 1
 * 2 5 6
 * 3 5 2
 * 4 5 3
 * 0
 */
/**
 * 模拟输入2 0序
 *
 *
 * 7 12
 * 0 1 12
 * 0 5 16
 * 0 6 14
 * 1 2 10
 * 1 5 7
 * 2 5 6
 * 2 4 5
 * 2 3 3
 * 3 4 4
 * 4 5 2

```

```
* 4 6 8
* 5 6 9
* 0
*/
int main() {
    // 模拟输入6
    int n; // 节点数
    cout << "输入节点数: ";
    cin >> n;

    // 初始化邻接矩阵, 也是二维矩阵
    graph.resize(n, vector<int>(n, INF));

    int m; // 边数
    cout << "输入边数: ";
    cin >> m;

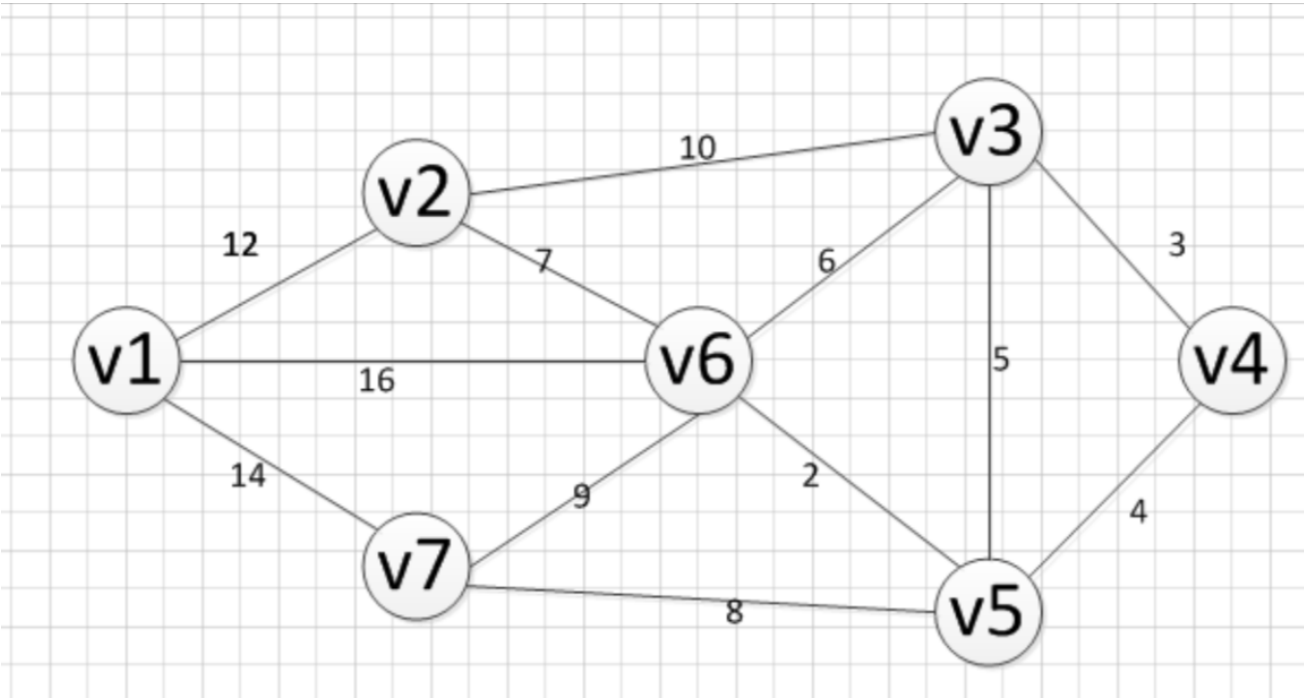
    cout << "输入边和权重: " << endl;
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        graph[u][v] = w;
        graph[v][u] = w; // 如果是有向图, 这行可以去掉
    }

    // 指定起始节点
    int start;
    cout << "输入起始节点: ";
    cin >> start;

    // 执行 Dijkstra 算法
    vector<int> distance, parent;
    dijkstra(start, distance, parent);

    // 模拟输出
    for (int i = 0; i < n; ++i) {
        if (i != start) {
            printShortestPath(start, i, distance, parent);
        }
    }

    return 0;
}
```



输入的图例

```
jzhou >> cd d:\code\c++\vscode\tulun\ ; if ($?) { g++ Dijkstra.cpp -o Dijkstra } ; if ($?) { .\Dijkstra }
输入节点数: 7 12
输入边数: 输入边和权重:
0 1 12
0 5 16
0 6 14
1 2 10
1 5 7
2 5 6
2 4 5
2 3 3
3 4 4
4 5 2
4 6 8
5 6 9
输入起始节点: 0
路径: 0 -> 1 最短路径长度: 12
路径: 0 -> 1 -> 2 最短路径长度: 22
路径: 0 -> 5 -> 4 -> 3 最短路径长度: 22
路径: 0 -> 5 -> 4 最短路径长度: 18
路径: 0 -> 5 最短路径长度: 16
路径: 0 -> 6 最短路径长度: 14
```

4. Floyd

Floyd 算法是一种动态规划算法，用于求解图中任意两点之间的最短路径。下面详细描述算法思想：

- 1. **初始化距离矩阵：** 首先，根据图的邻接矩阵初始化一个距离矩阵，其中`graph[i][j]`表示节点*i*到节点*j*的直接距离。如果节点*i*和节点*j*之间没有直接的边相连，则`graph[i][j]`的值设为无穷大（`INF`），而对角线上的元素初始化为0，表示节点到自身的距离。
- 2. **Floyd核心步骤：** 算法的核心是使用三层循环，逐个节点作为中间节点，更新距离矩阵。具体步骤如下：

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (graph[i][k] != INF && graph[k][j] != INF && graph[i][k] +
```

```

graph[k][j] < graph[i][j]) {
    graph[i][j] = graph[i][k] + graph[k][j];
}
}
}
}

```

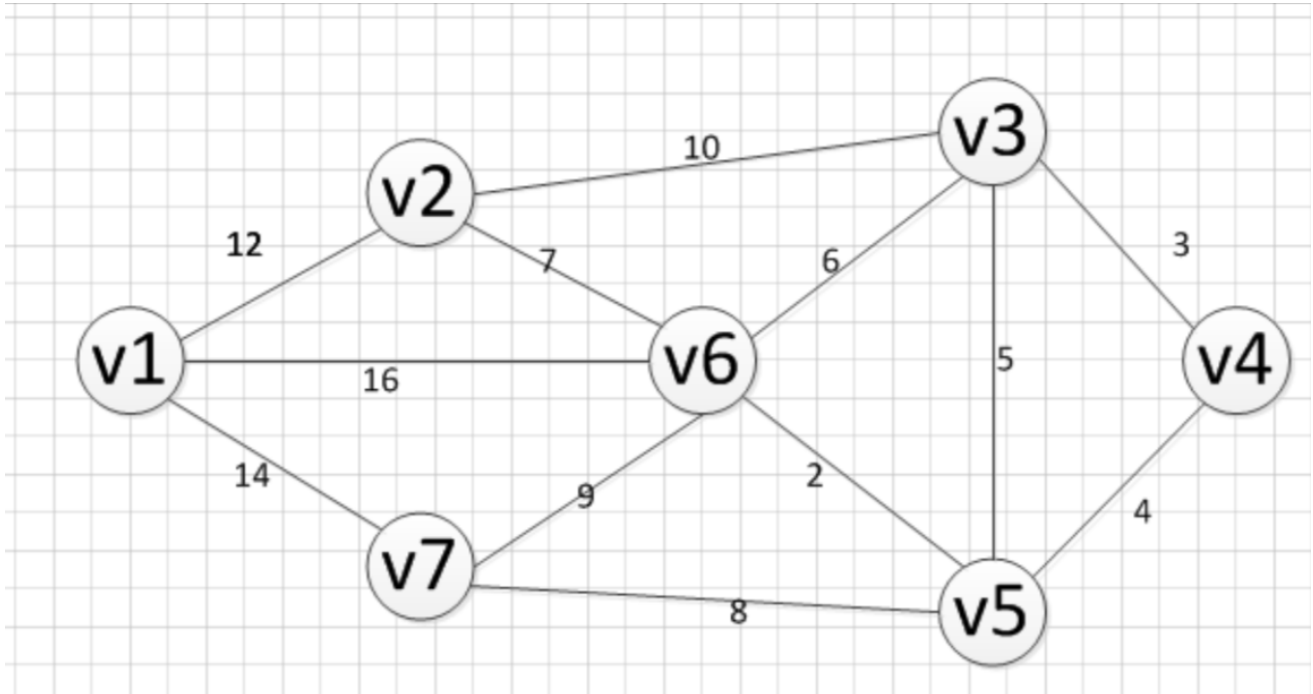
- 外层循环(k循环): 选择中间节点k。
- 中层循环(i循环): 遍历所有可能的起始节点i。
- 内层循环(j循环): 遍历所有可能的终点节点j。
- 如果经过中间节点k可以获得更短的路径 (即 $\text{graph}[i][k] + \text{graph}[k][j]$ 小于当前已知的 $\text{graph}[i][j]$) , 则更新距离矩阵。

这样, 经过三层循环的迭代, 距离矩阵中存储的值就是任意两点之间的最短路径长度。

- 输出结果:** 输出最终的距离矩阵, 其中 $\text{graph}[i][j]$ 即为节点i到节点j的最短路径长度。如果 $\text{graph}[i][j]$ 是无穷大(INF), 表示节点i和节点j之间没有路径。

Floyd算法通过逐步增加允许的中间节点, 不断更新最短路径矩阵, 直至考虑了所有节点作为中间节点的情况, 得到最终的最短路径矩阵。这种动态规划的思想使得算法具有全局性, 适用于任意两点之间的最短路径计算。

代码



输入的图例

```

#include <iostream>
#include <vector>
#include <climits>

using namespace std;

```

```

const int INF = INT_MAX;

// Floyd 算法
void floyd(vector<vector<int>>& graph) {
    int n = graph.size();

    // 逐个节点作为中间节点，更新距离矩阵
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                // 如果经过中间节点 k 能够获得更短的路径，则更新距离
                if (graph[i][k] != INF && graph[k][j] != INF && graph[i][k] +
graph[k][j] < graph[i][j]) {
                    graph[i][j] = graph[i][k] + graph[k][j];
                }
            }
        }
    }
}

// 输出距离矩阵
void printMatrix(const vector<vector<int>>& matrix) {
    int n = matrix.size();
    cout << "最短路径矩阵：" << endl;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (matrix[i][j] == INF) {
                cout << "INF ";
            } else {
                cout << matrix[i][j] << " ";
            }
        }
        cout << endl;
    }
}

/**
 * 0序
 *
 * 6 8
 * 0 1 4
 * 0 2 4
 * 1 2 2
 * 2 3 3
 * 2 4 1
 * 2 5 6
 * 3 5 2
 * 4 5 3
 */

int main() {
    // 模拟输入
    int n; // 节点数
    cout << "输入节点数：";
    cin >> n;

```

```

// 初始化邻接矩阵
vector<vector<int>> graph(n, vector<int>(n, INF));
for(int i = 0; i < n; i++) graph[i][i] = 0;
int m; // 边数
cout << "输入边数: ";
cin >> m;

cout << "输入边和权重: " << endl;
for (int i = 0; i < m; ++i) {
    int u, v, w;
    cin >> u >> v >> w;
    graph[u][v] = w;
    graph[v][u] = w; //如果是无向图就去掉这行
}

// 执行 Floyd 算法
floyd(graph);

// 模拟输出
printMatrix(graph);

return 0;
}

```

```

• jzhou >> cd "d:\code\C++\vscode\tulun\" ; if ($?) { g++ Floyd.cpp -o Floyd } ; if ($?) { .\Floyd }
输入节点数: 7 12
输入边数: 输入边和权重:
0 1 12
0 5 16
0 6 14
1 2 10
1 5 7
2 5 6
2 4 5
2 3 3
3 4 4
4 5 2
4 6 8
5 6 9
最短路径矩阵:
0 12 22 22 18 16 14
12 0 10 13 9 7 16
22 10 0 3 5 6 13
22 13 3 0 4 6 12
18 9 5 4 0 2 8
16 7 6 6 2 0 9
14 16 13 12 8 9 0

```

5. DFS

设图 G 的初态是所有顶点都“未访问过(False)”，在 G 中任选一个顶点 v 为源点,则广度优先搜索可定义为:

1. 首先访问出发点 v ,并将其标记为“访问过 (True)”;
2. 接着依次访问所有与 v 相邻的顶点 $w_1, w_2 \dots w_t$;
3. 然后依次访问与 $w_1, w_2 \dots w_t$ 相邻的所有未访问的顶点;
4. 依次类推, 直至图中所有与源点 v 有路相通的顶点都已访问过为止, 该过程称为从源点出发的一次先广搜索 (广度优先搜索)。

5. 此时，从 v 开始的搜索结束，若 G 是连通的，则遍历完成；否则在 G 中另选一个尚未访问的顶点作为新源点继续上述搜索过程，直到 G 中的所有顶点均已访问为止。

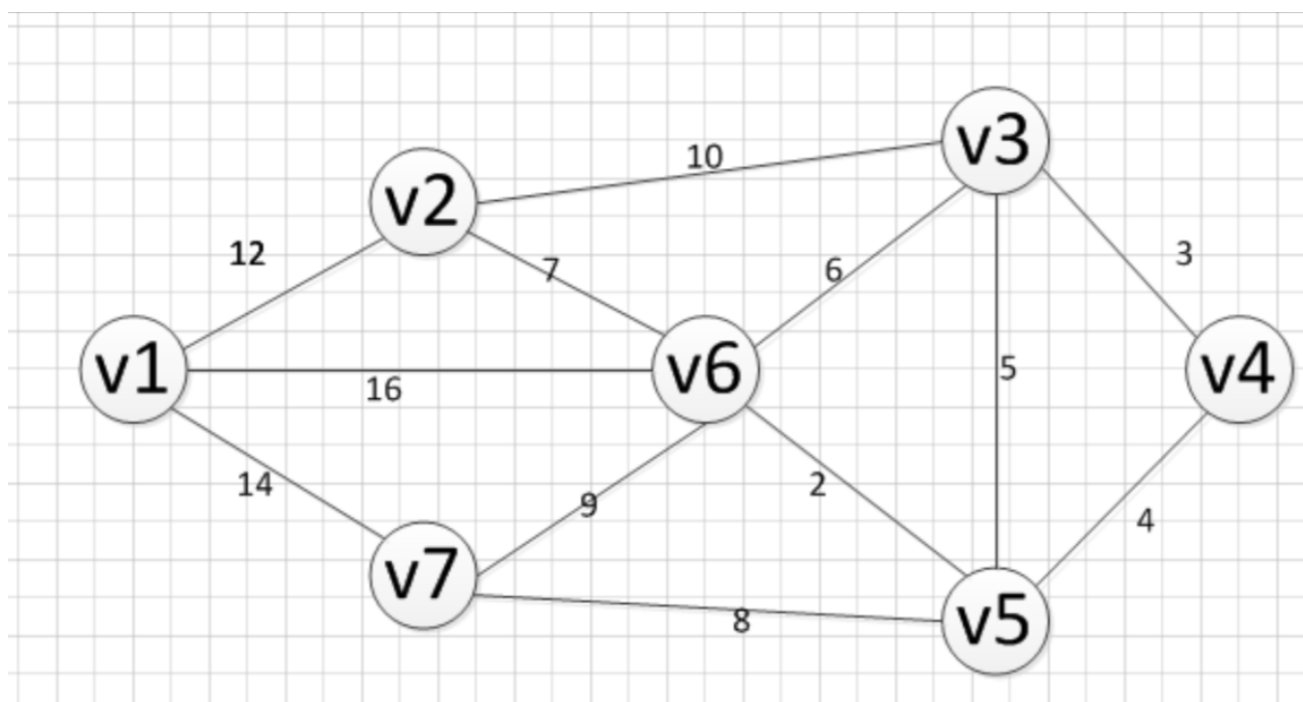
广度优先遍历特点:

- 尽可能横向上进行搜索，并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问,故称先广搜索或广度优先搜索。
- 搜索过程中，根据访问顺序给顶点进行的编号，称为**先广或广度优先编号**；
- 搜索过程中，根据访问顺序得到的顶点序列,称为**先广序列或BFS序列**。

时间复杂度:

邻接表 $O(N + E)$ $O(N + E)$ $O(N + E)$ ； 邻接矩阵 $O(N^2)$ $O(N^2)$ $O(N^2)$

代码



输入的图例

```
#include <iostream>
#include <vector>
#include <queue>
#include <unordered_set>

using namespace std;

// 边的结构体，包含邻接节点和权值
struct Edge {
    int to;
    int weight;
};

// 图的邻接表表示
vector<vector<Edge>> graph;
```



```
// BFS 遍历
void bfs(int start) {
    int n = graph.size();
    vector<bool> visited(n, false); // 记录节点是否被访问过

    queue<int> q;
    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        int current = q.front();
        cout << current << " ";
        q.pop();

        // 遍历当前节点的邻居
        for (const Edge& edge : graph[current]) {
            if (!visited[edge.to]) {
                q.push(edge.to);
                visited[edge.to] = true;
            }
        }
    }
}

int main() {
    // 模拟输入
    int n; // 节点数
    cout << "输入节点数: ";
    cin >> n;

    // 初始化邻接表
    graph.resize(n);

    int m; // 边数
    cout << "输入边数: ";
    cin >> m;

    cout << "输入边的连接关系和权值:" << endl;
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        // 添加边的信息到邻接表
        graph[u].push_back({v, w});
        graph[v].push_back({u, w});
    }

    // 指定起始节点
    int start;
    cout << "输入起始节点: ";
    cin >> start;

    // 执行 BFS 遍历
    cout << "BFS 遍历结果: ";
    bfs(start);

    return 0;
}
```

```
}
```

```
jzhou >> cd "d:\code\C++\vscode\tulun\" ; if ($?) { g++ BFS.cpp -o BFS } ; if ($?) { .\BFS }
输入节点数: 7 12
输入边数: 输入边的连接关系和权值:
0 1 12
0 5 16
0 6 14
1 2 10
1 5 7
2 5 6
2 4 5
2 3 3
3 4 4
4 5 2
4 6 8
5 6 9
输入起始节点: 0
BFS 遍历结果: 0 1 5 6 2 4 3
```

6. Huffman

Huffman树是一种特殊结构的[二叉树](#)，由Huffman树设计的二进制前缀编码，也称为Huffman编码在通信领域有着广泛的应用。在word2vec模型中，在构建层次Softmax的过程中，也使用到了Huffman树的知识。

问题描述

给定n个权值作为n个叶子节点，构造一棵二叉树，若这棵二叉树的带权路径长度达到最小，则称这样的二叉树为最优二叉树，也称为Huffman树。Huffman树是带权路径长度最小的二叉树。

节点的权越小，其离树的根节点越远。那么应该如何构建Huffman树呢？以上述报文为例，首先需要统计出每个字符出现的次数作为节点的权。

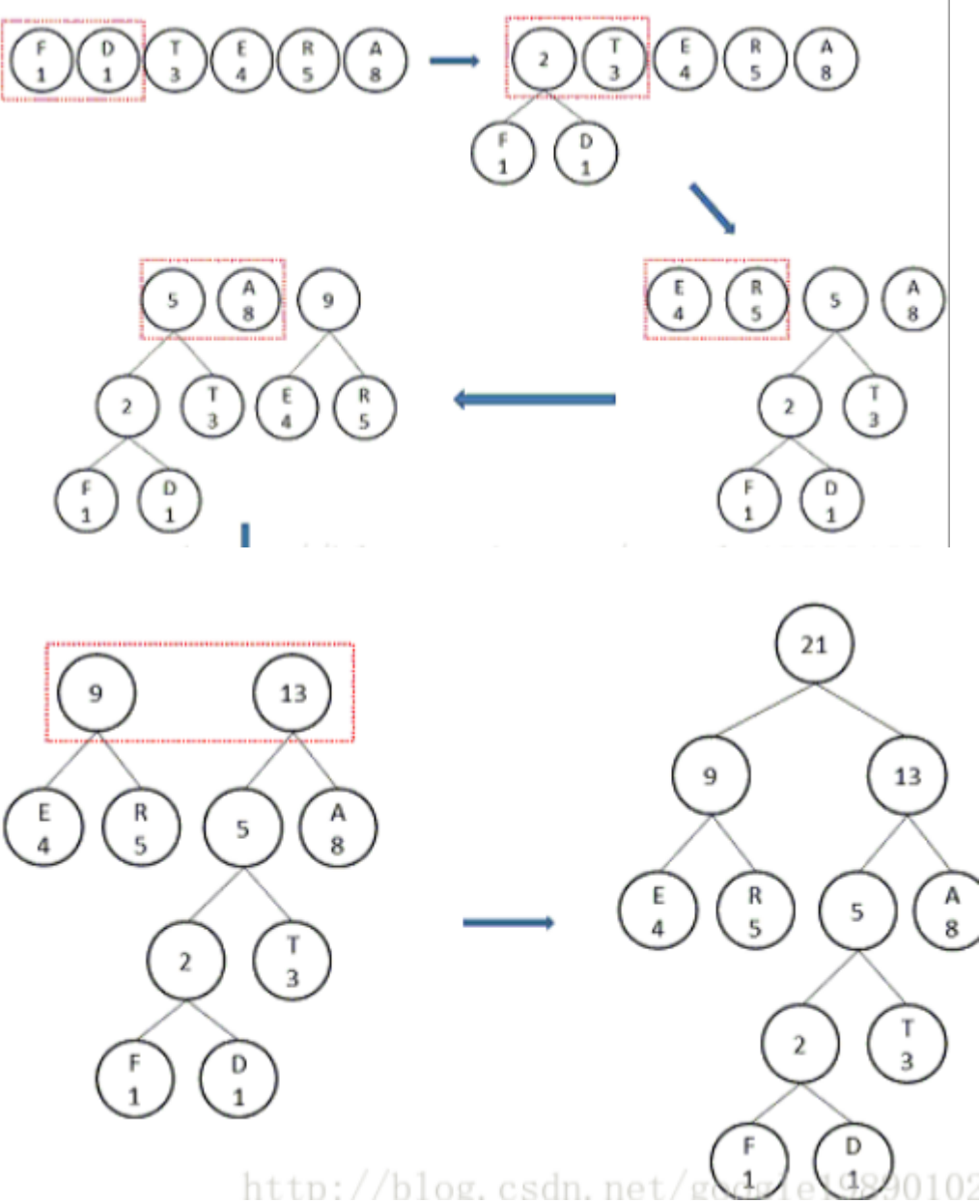
构建Huffman树：

- 重复以下的步骤：
 - 按照权值对每一个节点排序：D-F-T-E-R-A
 - 选择权值最小的两个节点，此处为D和F生成新的节点，节点的权重为这两个节点的权重之和，为2
- 直到只剩最后的根节点

```
A 8
D 1
E 4
F 1
R 5
T 3
```

给定这样6个权值的节点

按照上述的步骤，该Huffman树的生成过程为：



<http://blog.csdn.net/godlike1990102>

```
typedef struct
{
    /**
     * 记录节点
     */
    char ch;
    double weight; // 权重
    int parent, lchild, rchild; // 双亲
} HTNode, *HTree;
```

这颗树前n个节点存储的是叶子节点，带有权值 后面m-n个节点存储的是哈夫曼树的额外节点 每一个节点有父亲节点标号，同时有左右孩子标号，左右标号为0表示是没有左右孩子也就是叶子节点

代码

```

#include <iostream>
#include <algorithm>
#include <string>
using namespace std;
int m;
typedef struct
{
    /**
     * 记录节点
     *
     */
    char ch;
    double weight;           // 权重
    int parent, lchild, rchild; // 双亲
} HTNode, *HTree;

HTree HT;
// 8 7 19 2 6 32 3 21 10
void selet(int i, HTree HT, int &s1, int &s2)
{
    double w = 10000;
    for (int j = 1; j < i; j++)
    {
        /**
         * HT[j].parent == 0 就表示要么是一个孤立的点，要么是聚合的点的祖先，
         * 我们从所有的HT[j].parent == 0 找两个最小的点
         * 一旦找到了，我们会新建一个节点，
         * 这个新节点的左右儿子的parent指向新节点， 新节点的此时parent=0，等待下一
论查找
         *
         */
        if (w > HT[j].weight && HT[j].parent == 0)
        {
            w = HT[j].weight;
            s1 = j;
        }
    }
    for (int j = 1, w = 10000; j < i; j++)
    {
        /**
         * j != s1, 找第二小
         */
        if (w > HT[j].weight && j != s1 && HT[j].parent == 0)
        {
            w = HT[j].weight;
            s2 = j;
        }
    }
}

void CreatHuffman(HTree &HT, int n)
{
    /**
     * 这颗树前n个节点存储的是叶子节点，带有权值
     * 后面m-n个节点存储的是哈夫曼树的额外节点
     * 树用孩子父亲法表示

```

```

    * 每一个节点有父亲节点标号，同时有左右孩子标号，左右标号为0表示是没有左右孩子也
    就是叶子节点
    */
    if (n <= 1)
        return;    // 如果节点只有1个返回
    m = 2 * n - 1; // 哈夫曼的节点数为2*n-1

    HT = new HTNode[m + 1]; // 创建这颗树，&传引改变
    for (int i = 1; i <= m; i++)
    {
        HT[i].parent = HT[i].lchild = HT[i].rchild = 0; // 1到m 初始化
    }
    cout << "请输入每个结点和对应的权值\n";
    for (int i = 1; i <= n; i++)
    {
        // 输入数据
        cin >> HT[i].ch;
        cin >> HT[i].weight;
    }
    for (int i = n + 1; i <= m; i++)
    {
        // 进行构建树
        int s1, s2; // 找到最小的两个节点
        selet(i, HT, s1, s2);
        // 将节点指向父亲结点，父亲节点是新建的
        HT[s1].parent = HT[s2].parent = i;
        // 存储父亲节点的左右儿子
        HT[i].lchild = s1;
        HT[i].rchild = s2;
        // 父亲节点是新建的，新建的权值为两个儿子的和
        HT[i].weight = HT[s1].weight + HT[s2].weight;
    }
}

string HC[379]; // HC[i]为第i个字符的编码，一个HASH， 简单一点

bool Huffmancode(HTree HT, int n)
{
    /**
    * 编码从叶子节点找，叶子节点存储在0~n-1中，根据孩子父亲法，节点的父亲节点这样
    去编码
    * 由于从叶子往祖先编码，所以是逆序的
    * 编码遵循左0右1的规则
    */
    for (int i = 1; i <= n; i++)
    {
        int c = i, f = HT[i].parent;
        while (f != 0)
        {
            if (HT[f].lchild == c)
                HC[i] += '0'; // 左边为0
            else
                HC[i] += '1'; // 右边为1
            c = f;           // 双亲在往根节点探寻
            f = HT[f].parent;
        }
        reverse(HC[i].begin(), HC[i].end()); // 最后需要逆转
    }
}

```

```
    }
    return true;
}
// 6 a 0.07 b 0.09 c 0.12 d 0.22 e 0.23 f 0.27
/**
 * 6
 *
 * A 8
 * D 1
 * E 4
 * F 1
 * R 5
 * T 3
 *
 */

int main()
{
    int n;
    cout << "创建哈夫曼树" << endl;
    cout << "请输入元素个数\n";
    cin >> n;
    CreatHuffman(HT, n);
    cout << "创建成功" << endl;
    // post(HT,m);
    cout << endl;
    if (Huffmancode(HT, n))
        cout << "编码创建成功\n"
            << endl;
    cout << "编码如下" << endl;
    for (int i = 1; i <= n; i++)
    {
        cout << HT[i].ch << ' ';
        cout << HC[i] << endl;
    }
    // system("pause");
    return 0;
}
```

```
jzhou D:\code\C++\vscode\tulun 16ms 4:24 PM
• jzhou >> cd "d:\code\C++\vscode\tulun\" ; if ($?) { g++ Huffman.cpp -o Huffman } ; if ($?) { .\Huffman }
创建哈夫曼树
请输入元素个数
6
请输入每个结点和对应的权值
A 8
D 1
E 4
F 1
R 5
T 3
创建成功

编码创建成功

编码如下
A 11
D 1000
E 00
F 1001
R 01
T 101
```

